

Algorithm for file updates in Python

Project description

This project focused on updating an allow list of IP addresses for a healthcare company. The scenario involved identifying employees whose IP addresses were no longer authorized to access a restricted subnetwork containing personal patient records. To accomplish this task, I created a Python algorithm that reads the allow list from a file, converts the data into a list, and compares it against a remove list of unauthorized IP addresses. The algorithm removes any matching IP addresses, updates the allow list, and writes the revised data back to the file. This process helps ensure that only authorized employees can access restricted information.

Open the file that contains the allow list

“Allow_list.txt” is assigned to the import_file variable. A list of IP addresses no longer allowed is assigned to the variable remove_list. To open and read the file we start with a with statement that uses the open function to specify what file to open and how its treated, in this case the file is to be read so the second argument is “r”.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

.read() is used to read the imported file. Assigning file.read() to the variable ip_addresses stores the readable file in the variable. Print is used to display in the output.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)

```

Convert the string into a list

`.split()` function is used to split elements into a list or convert a string into a list. Reassigning `ip_addresses.split()` to the variable `ip_addresses` converts that string into a list.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)

```

Iterate through the remove list

A for loop allows iteration through usernames in a list. The variable for the for loop is “element” instead of “i” and uses the “in” operator to specify to run the loop for every item in the sequence.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)

```

Remove IP addresses that are on the remove list

A conditional statement evaluates code to see if it meets specified conditions. If starts a conditional statement. `.remove` function removes the first occurrence of an element in a list. Adding `.remove` to `ip_addresses` and passing element through removes the occurrences of `remove_list` from `ip_addresses`.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a List
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)

```

Update the file with the revised list of IP addresses

`.join` concatenates elements into a string. “ ” specifies to separate the elements in the list by a space. Assigning “ “.join(ip_addresses) to the variable `ip_addresses` stores the concatenated string separated by spaces into the variable. The file is then opened again but this time the second argument is “w” which means to write to the file. `file.write(ip_addresses)` rewrites the file.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)

```

Summary

In this activity, I used Python to automate the process of updating an allow list. By reading data from a file, converting it into a list, iterating through the entries, and removing unauthorized IP addresses, I was able to efficiently update access permissions. The algorithm then converted the updated list back into a string and wrote the revised data to the file. This project strengthened my understanding of file handling, loops, conditional statements, and list methods in Python. It also demonstrated how automation can help security professionals manage access controls more effectively.